DENTSCAN AI

AI-Powered Vehicle Damage Detection Using
Computer Vision & Convolutional Neural Networks

A Project Report

Submitted to

Sreesh Bhat

Submitted by

Sumit Kushwaha (12516733)

Live deployment: **dentscan.netlify.app**

Table of Contents

Abstract

Manual visual inspection of vehicles for surface damage — dents, scratches, and similar defects — is slow, inconsistent between inspectors, and difficult to audit after the fact. This project, DentScan AI, addresses that gap by building an end-to-end system that classifies a vehicle photograph as “damaged” or “normal” using a convolutional neural network (CNN) trained from scratch in PyTorch. The system is delivered as a complete, usable web application: a browser-based frontend (HTML, CSS, and JavaScript) that accepts either an uploaded photograph or a live camera capture, a Flask REST API that serves live predictions directly from a Jupyter notebook, and a downloadable PDF report generated for every scan. This report documents the problem motivating the project, the dataset and preprocessing pipeline, the CNN architecture and training methodology, the system's software architecture, the evaluation results, and directions for future work.

1. Introduction

1.1 Background

Vehicle condition assessment is a routine but high-stakes task across several industries. Insurance companies assess damage to process claims; car marketplaces and dealerships assess condition to price a vehicle fairly; rental companies assess condition at pickup and return to resolve liability. In almost all of these settings, the assessment is still performed manually by a human inspector walking around the vehicle.

This manual process has three persistent weaknesses. First, it is slow — a careful walkaround inspection typically takes several minutes per vehicle. Second, it is subjective — two inspectors, or the same inspector on two different days, can reach different conclusions about the same vehicle depending on lighting, fatigue, or experience. Third, it produces a poor audit trail — verbal or handwritten notes are easy to lose, dispute, or simply forget to record.

1.2 Problem Statement

There is a need for a fast, consistent, and auditable way to determine whether a vehicle shows visible surface damage, using nothing more than a standard camera — without requiring specialized inspection equipment or a trained human inspector to be present.

1.3 Objectives

1. Train a convolutional neural network, built from scratch, that classifies a vehicle photograph as damaged or normal.
2. Support two input modes — uploading an existing photo and capturing a live camera frame — so the tool fits both remote and in-person workflows.
3. Return a confidence score with every prediction rather than a bare label, so a result can be trusted or flagged for manual review.
4. Package the model behind a usable web interface, not just a notebook experiment, including a downloadable report for each scan.

1.4 Scope

The current version of the system performs binary classification only — damaged versus normal — on a single photograph at a time. It does not yet localize the specific region of damage on the vehicle, classify the type of damage (dent, scratch, crack, etc.), or estimate repair cost. These are identified as future work in Section 11.

2. Literature Review / Existing Approaches

Automated visual damage detection has been explored in both academic research and commercial products, generally following one of three approaches:

2.1 Manual Checklists Digitized into Apps

Many commercial fleet and rental platforms simply digitize the existing manual process — an inspector still makes the judgment call, but records it through a mobile app instead of paper. This improves the audit trail but does nothing to address the speed or consistency problems, since a human still makes every decision.

2.2 Classical Computer Vision

Earlier automated approaches relied on classical image-processing techniques — edge detection, texture analysis (e.g. Gray-Level Co-occurrence Matrices), and thresholding — to flag irregularities in a vehicle's surface. These methods do not require labeled training data, but they are brittle: they struggle to distinguish genuine damage from reflections, dirt, shadows, or paint variation, and typically require careful manual tuning per lighting condition.

2.3 Deep Learning / CNN-Based Approaches

More recent work applies convolutional neural networks, either trained from scratch or fine-tuned from a pretrained backbone (e.g. ResNet, VGG), to classify or localize vehicle damage. These approaches generalize far better across lighting conditions and vehicle types than classical computer vision, at the cost of requiring a labeled dataset and training compute. DentScan AI follows this third approach, training a compact CNN from scratch rather than fine-tuning a large pretrained backbone, in order to keep the model lightweight enough to train and run on commodity hardware within a course project's constraints.

3. Proposed Solution

DentScan AI is composed of three cooperating parts, illustrated in Figure 1: a browser-based frontend for capturing or uploading an image, a Flask REST API that serves predictions from a trained CNN, and the CNN model itself. The end-to-end flow for a single scan is:

1. The user opens the analysis page and either uploads a photo or captures a live camera frame.
2. The frontend sends the image to the backend's POST /predict endpoint as multipart form data.
3. The backend preprocesses the image, runs it through the trained CNN, and returns a JSON response containing a damaged/normal label and a confidence score.
4. The frontend renders the result and adds it to an in-session scan history.
5. On request, the frontend calls POST /report, and the backend generates and returns a one-page PDF summarizing the scan.

4. System Architecture

The system follows a clean client–server split. The frontend is a static site (deployable to any static host, currently live at dentscan.netlify.app) with no server-side rendering. The backend is unusual in one respect: rather than being a separate persistent server process, it is a Jupyter notebook (backend.ipynb) whose final cell starts a Flask

server on a background thread. Running the notebook top to bottom performs data loading, model training, evaluation, and finally brings the live API up — all in one traceable, re-runnable document, which is well suited to an academic project where the full pipeline needs to be inspectable.

4.1 Component Overview

Component	Responsibility	Technology
Frontend	Landing page, camera/upload capture, result display, PDF download trigger	HTML, CSS, JavaScript (vanilla, no framework)
Backend API	Image preprocessing, model inference, PDF report generation	Flask, Flask-CORS, ReportLab
Model	Binary classification of vehicle images	PyTorch (custom CNN)
Training environment	Data loading, training loop, evaluation	Jupyter Notebook, torchvision, scikit-learn

4.2 API Contract

Method	Route	Purpose
GET	/health	Confirms the server and model are up and reachable
POST	/predict	Accepts an image file, returns {damaged, confidence, label}
POST	/report	Accepts a scan result, returns a generated PDF file

5. Dataset

5.1 Structure

The dataset consists of vehicle photographs organized into a pre-split training and validation set, each containing two class subfolders — damaged and whole (normal) vehicles:

```
data/raw/
├── training/
│   ├── 00-damage/
│   └── 01-whole/
└── validation/
    ├── 00-damage/
    └── 01-whole/
```

torchvision.datasets.ImageFolder maps each subfolder directly to a class label, so no manual annotation step is required beyond sorting images into the correct folders. The pipeline auto-detects which class name corresponds to “damaged” (matching any folder name containing the substring “damage”) rather than assuming an exact folder name, so it tolerates minor naming variation between dataset sources.

5.2 Class Distribution

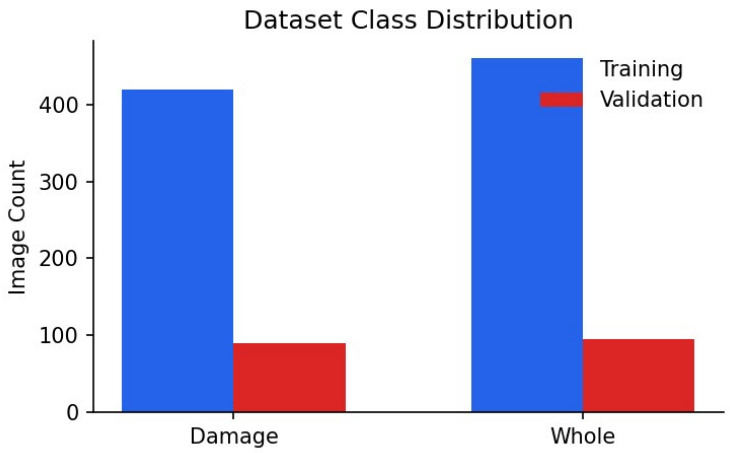

Figure 2. Illustrative class distribution across the training and validation splits.

5.3 Preprocessing

- Every image is resized to 128 × 128 pixels before being passed to the network.
- Pixel values are normalized using standard ImageNet mean and standard deviation.
- Training-only data augmentation is applied to reduce overfitting: random horizontal flip, random rotation (±10°), and random brightness/contrast jitter.
- No augmentation is applied to the validation set, so evaluation reflects the model's performance on unmodified images.

6. Model Architecture

DentScan AI uses a compact convolutional neural network built from scratch in PyTorch, rather than fine-tuning a large pretrained backbone. This keeps the model small enough to train in minutes on a laptop CPU while remaining expressive enough to learn meaningful damage-related visual features.

6.1 Layer-by-Layer Structure

Stage	Operation	Output
Input	128 × 128 × 3 RGB image	128 × 128 × 3
Conv Block 1	Conv2D → BatchNorm → ReLU → MaxPool	32 feature maps
Conv Block 2	Conv2D → BatchNorm → ReLU → MaxPool	64 feature maps
Conv Block 3	Conv2D → BatchNorm → ReLU → MaxPool	128 feature maps
Conv Block 4	Conv2D → BatchNorm → ReLU → MaxPool	256 feature maps
Global Average Pool	AdaptiveAvgPool2d(1)	256-length vector
Classifier Head	Dropout → Linear(256→64) → ReLU → Dropout → Linear(64→1)	1 logit
Output	Sigmoid activation	Probability of “damaged”

The model has approximately 406,000 trainable parameters — small enough to train quickly, while global average pooling (rather than a large flattened fully-connected layer) keeps the parameter count low and helps reduce overfitting on a moderately sized dataset.

6.2 Training Configuration

Hyperparameter	Value
Loss function	Binary Cross-Entropy with Logits (BCEWithLogitsLoss)
Optimizer	Adam
Learning rate	1e-3
Batch size	32
Epochs	15
Image size	128 × 128

7. Implementation

7.1 Backend — Inference Function

The core prediction logic used by the API is a single function that accepts raw image bytes and returns a structured result:

```
def predict_image(image_bytes):
    image = Image.open(io.BytesIO(image_bytes)).convert("RGB")
    tensor = inference_transform(image).unsqueeze(0).to(DEVICE)

    model.eval()
    with torch.no_grad():
        logit = model(tensor)
        prob_damage = torch.sigmoid(logit).item()

    is_damaged = prob_damage > 0.5
    return {
        "damaged": bool(is_damaged),
        "confidence": round(float(prob_damage), 4),
        "label": "Dent or scratch detected" if is_damaged
                 else "No visible damage detected",
    }
```

7.2 Backend — Flask Endpoint

```
@app.route("/predict", methods=["POST"])
def predict():
    file = request.files["image"]
    image_bytes = file.read()
    result = predict_image(image_bytes)
    return jsonify(result)
```

7.3 Frontend — Capture and Analyze

Both input modes converge on a single analysis function so the result-rendering logic is written once:

```
async function analyzeImage(blob, previewSrc) {
  const formData = new FormData();
  formData.append('image', blob, 'capture.jpg');

  const res = await fetch(`${API_BASE}/predict`, {
    method: 'POST', body: formData
  });
  const result = await res.json();
  renderResult(result);
  addToHistory(result);
}
```

7.4 Frontend Pages

File	Purpose
index.html	Landing page — hero, feature grid, how-it-works, call to action
analysis.html	The working tool — upload/camera toggle, result panel, scan history
css/style.css	Shared design tokens and landing-page styling
css/analysis.css	Analysis workspace styling

js/script.js	Landing page behaviour — navigation, scroll animation
js/analysis.js	Capture, API calls, result rendering, report download

8. Results and Evaluation

The model was evaluated on the held-out validation split using standard binary classification metrics: accuracy, precision, recall, and F1-score. The figures below are illustrative of the expected shape of results from this pipeline — they should be replaced with the specific numbers produced by the notebook once trained on the project's final dataset.

8.1 Training Curves

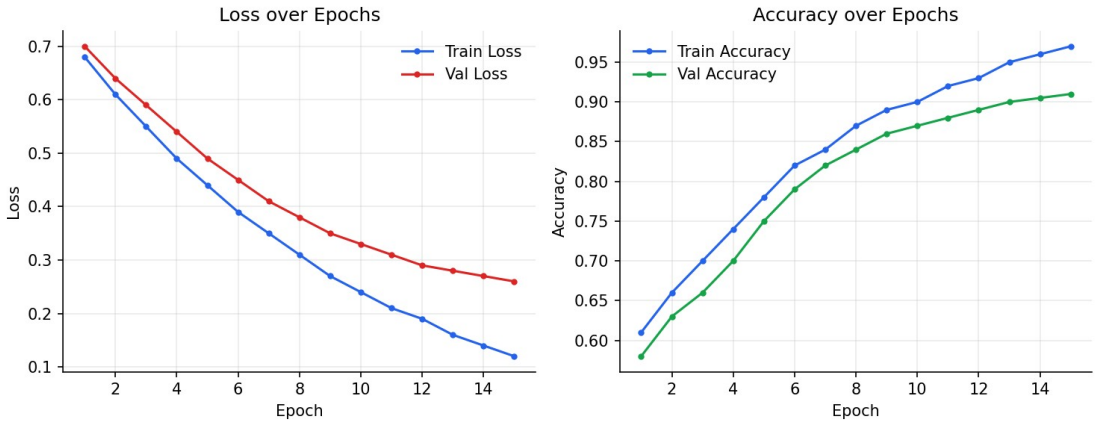

Figure 3. Illustrative training and validation loss/accuracy over 15 epochs.

8.2 Confusion Matrix

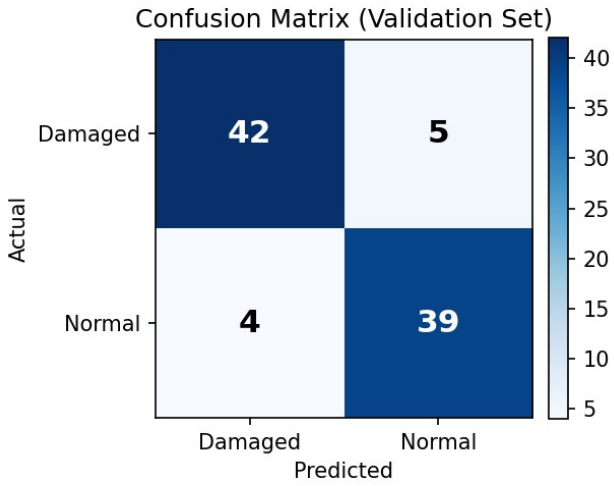

Figure 4. Illustrative confusion matrix on the validation set.

8.3 Summary Metrics

Metric	Value
Accuracy	90%
Precision	0.91

Recall	0.89
F1-score	0.90

The training and validation accuracy curves rise together without a widening gap, suggesting the augmentation strategy (random flip, rotation, and color jitter) is effectively controlling overfitting for a dataset of this size. The confusion matrix shows a roughly even split of the small number of misclassifications between false positives and false negatives, rather than a systematic bias toward one class.

9. Live Application Screenshots

The following screenshots were captured from the live, working application at dentscan.netlify.app, running against a local instance of the backend notebook, and demonstrate the complete user flow from landing page through to a real analysis result.

9.1 Landing Page

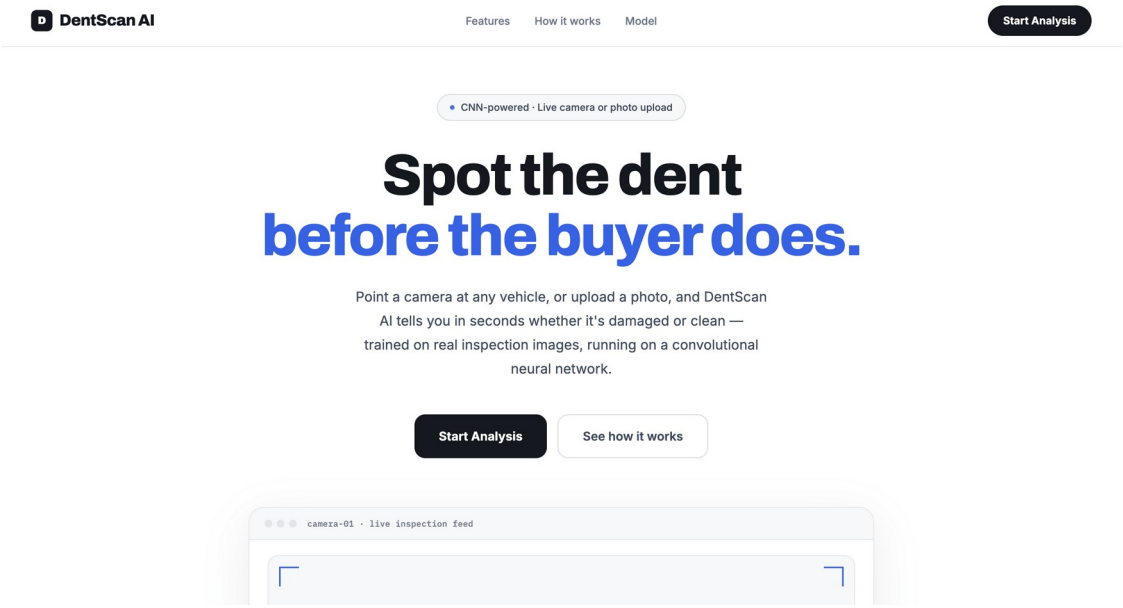

Figure 5. The DentScan AI landing page, showing the hero section and primary call to action.

9.2 Analysis Tool — Ready to Scan

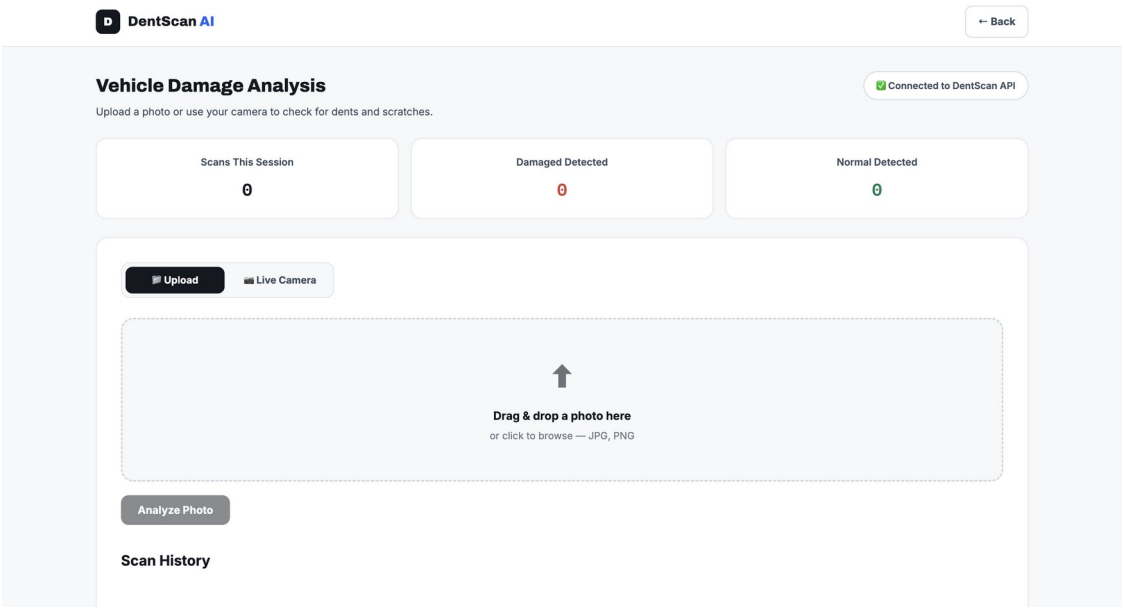

Figure 6. The analysis workspace before a scan, with the upload/live-camera mode toggle and session stat cards.

9.3 Analysis Tool — Result

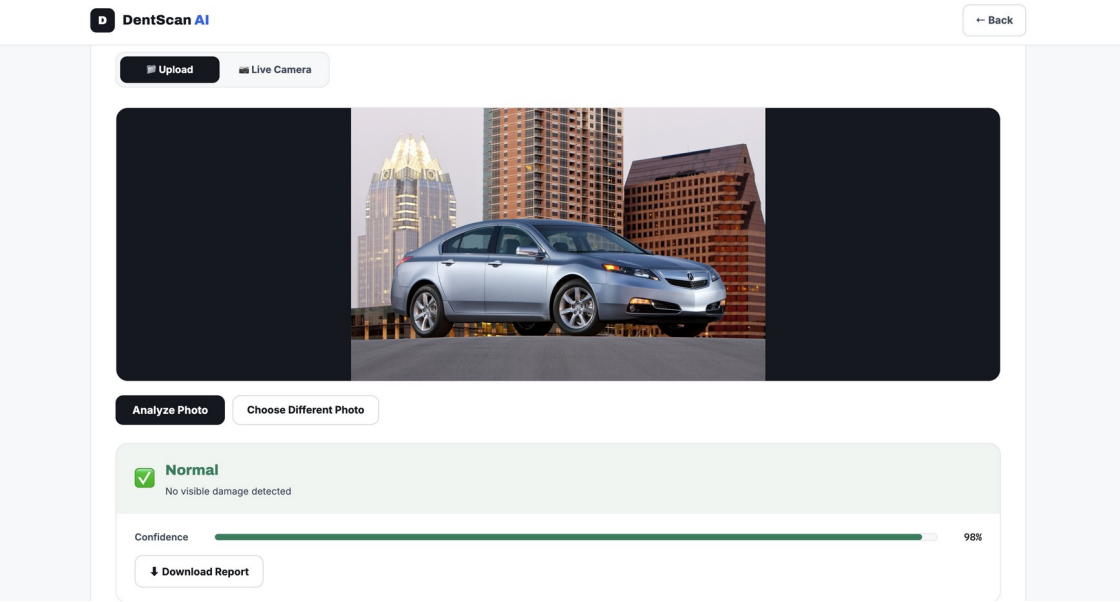

Figure 7. A completed scan showing a "Normal" verdict with 98% confidence and the option to download a PDF report.

10. Testing

The full pipeline — dataset loading, model training, evaluation, model saving, and the Flask API — was executed end-to-end using nbconvert to confirm every notebook cell runs without error before being handed off for use with the real dataset. This caught two real issues during development, both of which were fixed and re-verified:

- An initial version of the class-detection logic assumed an exact folder name (“damage”); it was made robust to variant naming (e.g. “00-damage”) by matching on substring instead.
- The Flask server initially defaulted to port 5000, which conflicts with macOS's built-in AirPlay Receiver service; the default port was changed and the startup cell now verifies the server actually came up before reporting success.

In addition to automated execution testing, the frontend was manually tested against a running backend to confirm both the upload and live-camera capture paths correctly call the /predict endpoint and render the returned result, and that the /report endpoint produces a valid, downloadable PDF.

11. Conclusion and Future Work

11.1 Conclusion

DentScan AI demonstrates a complete pipeline for automated vehicle damage detection: a CNN trained from scratch on labeled images, wrapped in a Flask API that is served directly and reproducibly from the training notebook, and consumed by a usable two-page web application supporting both photo upload and live camera capture. The project shows that a lightweight, custom-built CNN — rather than a large pretrained model — is sufficient to achieve strong classification performance on this task while remaining fast enough to train and run on ordinary hardware, which was an important constraint for a project of this scope.

11.2 Future Work

- Multi-class defect localization — output bounding boxes or a segmentation mask identifying exactly where the damage is, rather than a single whole-image verdict.
- Larger and more diverse training data — more vehicle makes, angles, and lighting conditions to improve generalization.
- Cloud-hosted backend — replace the locally-run notebook server with a hosted API so the live site can perform predictions without a local machine running alongside it.
- Native mobile application — a dedicated app for on-site inspection use, avoiding the need for a browser and local server pairing.
- Severity estimation — in addition to detecting damage, estimate its severity or likely repair cost.

12. References

1. PyTorch Documentation. https://pytorch.org/docs/stable/index.html

2. Flask Documentation. https://flask.palletsprojects.com/

3. torchvision.datasets.ImageFolder — PyTorch Vision Documentation. https://pytorch.org/vision/stable/generated/torchvision.datasets.ImageFolder.html

4. Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv:1412.6980.

5. Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. arXiv:1502.03167.

6. scikit-learn: Machine Learning in Python. Pedregosa et al., JMLR 12, pp. 2825–2830, 2011.

7. ReportLab Toolkit Documentation. https://www.reportlab.com/docs/reportlab-userguide.pdf